

package or as lexically scoped before it is used. In fact, every program should start with a 'use strict' and it is very important that the variables are declared in the smallest possible scope, so as to minimise the 'surprising' outputs.

② Modularising

Impatience is an integral characteristic of every programmer. Whenever some task is assigned, we dive into implementing the requirement and start coding pages and pages in a single block/routine. This is a bad habit. It is always worth spending a good amount of time in planning and breaking down your task into smaller atomic routines and giving it a proper name, because *that* is the most crucial part.

➤ Divide and rule



Half your work is done, if this is done perfectly. Writing a single function for eating, exercising and sleeping is also a bad habit. Each piece of code should do one thing and do it well. Furthermore, this practice will make the program more maintainable, and make the debugging and testing much easier. A hash of named arguments are much better than a simple array if you have more than three parameters to be passed to a subroutine. It replaces the need to remember the order of the parameters you pass to the routines.

③ Self-documentation

This is nothing but making the code easily readable. It can be achieved by giving the variables, functions, files, etc., a meaningful name. Don't create confusion while naming variables. The name should reflect what it holds.

☞ Clarity makes life easier



This will increase the clarity in your program, which eventually reduces confusion. Since the errors are more obvious, you are less likely to make mistakes. The basic idea of a language, be it spoken or programming, is to communicate. So make your program speak for itself.

④ A bit of discipline

Presentation has an enormous impact. Each of the nested blocks should be properly intended. If you don't want to spend time on these cosmetic touches, you can use *Perl::Tidy* which can be downloaded from CPAN. It parses and beautifies your code. This can be even configured according to the style you prefer. Another point on this topic is the naming of your variables and routines. All the names should be largely self-explanatory. Code should read like prose and not like a puzzle.



➤ Cosmetic touches are needed

⑤ API/module designing

This is all about the designing of the programmer's interface. It requires both experience and creativity. An improper API design will end up in unexpected performance degradation and reduce the maintainability of the system. One good practice of API designing is to write the sample codes that will use the module before writing the module itself. This will help in figuring out how the module should work.

Some of the characteristics of a good API are: easy to use, easy to learn, easy to extend, hard to misuse, easy to read and maintain code that uses it, appropriate to audience and sufficiently powerful to satisfy the requirements. Starting an API design with a small spec will be ideal, so that it's easy to modify. I strongly recommend reading 'Perl Best Practices' by Damian Conway before a single line of code is written.



➤ Plug and play



By: Arshad Mohamed

The author is a Web2.0 developer currently working with Genseq, Malaysia as a developer for an online Health Reporting System. He has worked over four years as programmer/developer, and mostly uses Perl. His other expertise includes Ajax, MySQL, Linux, Apache and JavaScript.